BHP/1 — HTTP in binary frames

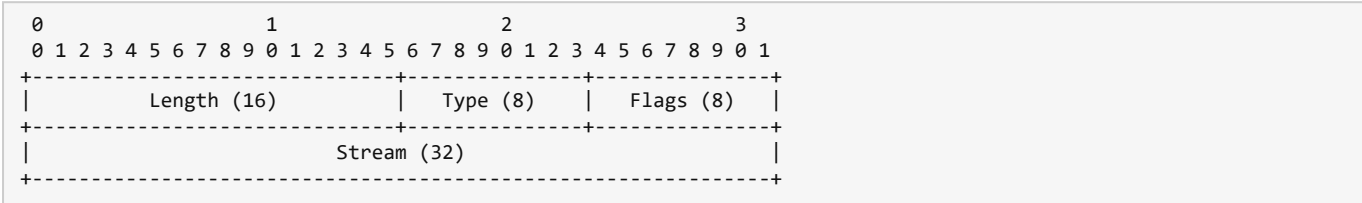
The words MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119.

1. Model

BHP carries HTTP's request/response semantics over one TCP connection as a sequence of *frames*. The client connects and sends REQUEST frames; the server answers each with one RESPONSE frame followed by zero or more DATA frames. There is no handshake: the first byte on the wire is the first byte of a frame header. The connection stays open after a response. A client sends all of its requests on it and closes it once it has every answer. All integers are unsigned and big-endian. No port is assigned; examples use 9000.

2. Frame header

Every frame is an 8-byte header followed by exactly `Length` bytes of payload.



Field	Meaning	Why this width
Length (16)	Payload bytes, 0–65 535. The header is not counted.	Any legal frame fits a 64 KiB buffer, so a receiver's memory is bounded with nothing to negotiate. A larger body is just more DATA frames: 8 bytes per 64 KiB, 0.012 %.
Type (8)	What the payload is (§3).	Three used, 253 left for later versions.
Flags (8)	Per-type bits. Undefined bits MUST be sent as 0 and MUST be ignored on receipt.	One bit used; a whole byte keeps the header aligned.
Stream (32)	The request/response exchange this frame belongs to.	Lets request N+1 leave before response N has finished, which is why one connection is enough. 2^{32} ids last five days at 10 000 requests/s, so ids are never recycled.

Against HTTP/2's 24/8/8/31 in 9 bytes: Length drops to 16 bits because HTTP/2 needs a SETTINGS parameter to say how much of its 24 a receiver will really take (16 KiB unless raised); Stream takes the whole word because the reserved bit has no job here; and 8 bytes puts every field on its natural alignment.

Length comes first and means the same thing for every type, so a receiver can always find the next frame without understanding the current one. There is no version field: that property, with §5 rule 1, is the versioning mechanism.

The client chooses each request's Stream id: non-zero, and never reused on a connection (count up from 1). The server copies it into every frame of the response, so it keeps no table. Stream 0 is reserved for future connection-level frames.

3. Frame types

Type	Name	Sent by	Payload	Flags
`0x01`	REQUEST	client	field block (§4)	none
`0x02`	RESPONSE	server	field block (§4)	END
`0x03`	DATA	server	body bytes	END

END (`0x01`) marks the last frame of a response. A response is exactly one RESPONSE frame followed by zero or more DATA frames, all carrying the request's Stream id; the last of them, and only the last, has END set. The body is the concatenation of the DATA payloads. END is what delimits it; `content-length` is a hint. A response without a body is a lone RESPONSE frame with END.

The frames of one response arrive in order. Frames of different responses MAY be interleaved, and responses MAY finish in any order: a client with several requests outstanding MUST sort frames by Stream.

4. Field block

A field block is a sequence of fields that fills the payload exactly. A field is a name and a value; its first byte says how the name is given.

```
name by number:  | Index (8) = 1..255 | VLen (16) | Value (VLen bytes) |
name spelled:    | 0x00 | NLen (8) | Name (NLen bytes) | VLen (16) | Value (VLen bytes) |
```

Index	Name	Index	Name
1	`:method`	6	`:status`
2	`:path`	7	`content-type`
3	`host`	8	`content-length`
4	`user-agent`	9	`server`
5	`accept`	10	`last-modified`

Names are lowercase ASCII and NLen is at least 1. Values are raw bytes with no terminator. A table name that is spelled out means the same as its number. Order does not matter. A receiver MUST ignore a field whose Index is above 10 or whose name it does not recognise; VLen always says how far to skip.

A REQUEST MUST contain exactly one `:method` and exactly one `:path`. `:method` is `GET` or `HEAD`. `:path` begins with `/`, is percent-encoded as in an HTTP URL, and may end in `?query`. A RESPONSE MUST contain exactly one `:status`: three ASCII digits with HTTP's meanings.

5. Receiving what you did not expect

1. **Unknown Type.** The receiver MUST read and discard `Length` bytes and carry on with the next frame. It MUST NOT reply and MUST NOT close. Version 2 will add frame types, and version 1 peers will step over them.
2. **Known type, wrong place** — DATA or RESPONSE arriving at a server, REQUEST arriving at a client, or any frame for a stream on which the client has no request outstanding. Discard it as in rule 1.
3. **Malformed REQUEST** — a field runs past the end of the payload, NLen is 0, `:method` or `:path` is missing or repeated, `:path` does not begin with `/`, or Stream is 0. The server MUST answer on the same Stream with a complete `400` response and MUST keep the connection open. Length was honoured, so the framing is intact.
4. **Malformed response** — a bad field block, no `:status`, a second RESPONSE, or DATA before RESPONSE. The client treats that request as failed and SHOULD close the connection.
5. **End of connection.** Closing between frames is always legal. If the connection ends inside a frame, the frame is void; a server SHOULD send a `400` on that frame's Stream (0 if the header itself was cut short) and then close. A client counts every unanswered request as failed.

6. Server behaviour

The server maps `:path` to a file under its root: drop `?query`, percent-decode, resolve what is left relative to the root. A directory means its `index.html`. It MUST NOT serve anything outside the root, however the path is spelled.

Status	When
`200`	The file exists. GET: the body follows in DATA frames. HEAD: the same fields, and the RESPONSE frame carries END.
`400`	Malformed REQUEST (§5 rule 3), or a bad percent-escape in `:path`.
`404`	Nothing to serve at that path.
`405`	`:method` is neither `GET` nor `HEAD`.

An error response carries a short `text/plain` body that says what went wrong.

7. Example

`GET /` for a five-byte file, as hex. Each line is one frame: the 8-byte header, then the payload.

```
00 0a 01 00 00 00 00 01   01 00 03 47 45 54  02 00 01 2f    REQUEST    :method=GET  :path=/
00 06 02 00 00 00 00 01   06 00 03 32 30 30                RESPONSE   :status=200
00 05 03 01 00 00 00 01   68 65 6c 6c 6f                   DATA, END  "hello"
```